

1. Project overview

Mini Cloud: resource-aware scheduling and worker recovery

Midterm Project Report | Cumulative progress through Week 7
Prepared for Shamshad Ansari (name inferred from repository commits)
Evidence snapshot: 6 October 2026 | Commit: 50d05f0

Purpose. Mini Cloud studies how a small private-cloud controller can admit CPU/memory requests, select workers, launch replicated workloads, and restore desired replicas after worker failure. The central problem is making placement and lifecycle decisions observable and reproducible while avoiding resource overcommitment.

Objectives. (1) Build a C++23/Linux admission and scheduling foundation. (2) Compare deterministic first-fit and least-loaded dominant-resource policies. (3) Implement controller/worker communication, supervision, registration, heartbeats, and remote launch. (4) Extend placement to replicas, explicit stop, failure detection, and recovery. (5) Enforce admitted CPU/memory limits. (6) Evaluate placement success, resource balance, scheduling cost, and recovery time with reproducible workloads.

Expected final product. A runnable controller and worker-agent prototype, automated tests, versioned experiment scenarios, raw lifecycle traces, a policy/recovery analysis, and documented operational limits. This is a systems-study prototype for trusted workloads, rather than a production cloud service.

Evidence and scope

Repository: <https://github.com/shamshad-ansari/mini-cloud>

Source references throughout this report are relative to the repository root. The attached evidence folder contains the test log, scheduler benchmark source, raw CSV, and report generator.

The cumulative account below groups work by milestones because exact semester-week dates are unavailable. Git commits establish implementation dates, not the dates of every design activity. The original approved proposal/AS1.pdf and an individual contribution log were not available in this workspace. Proposal comparisons are therefore explicitly reconstructed from supplied feedback and local issue plans; individual authorship and exact weekly allocations should be checked against the original submission.

Evidence levels are kept separate: inspected implementation; tests executed against real local OS processes; scheduler simulation using synthetic capacities; and planned experiments. No measurements from separate physical worker nodes are claimed.

2. Work completed: Weeks 1-7

Cumulative milestones, part A

A. Scope and design. The supplied proposal feedback describes a private-cloud systems study with scheduling baselines, failure scenarios, and measurable outcomes. Local issue drafts 001-010 refine that direction into dependency-ordered implementation slices: admission, policy CLI, supervision, registration, remote launch, replicas, recovery, cgroup enforcement, then experiments. Output: a concrete prototype architecture and acceptance criteria. Limitation: the decision record linked by issues/README.md is absent, so its contents are not asserted.

B. Resource-admission foundation. `src/resources.cpp` and `include/mini_cloud/resources.hpp` implement integer CPU millicores and memory MiB, positive capacities, valid reservations, checked unsigned-64-bit additions, and CPU/memory admission decisions. `src/main.cpp` exposes a terminal tracer with accepted, no-placement, and invalid-input outcomes. Output: executable `resource_admission_tracer` plus `resource_tests`. Contribution: later schedulers and the controller share the same admission logic, reducing inconsistent overcommitment checks. Commit 96c3e1e, 14 September 2026, establishes the initial buildable tracer.

C. Explicit scheduling policies. `src/scheduler.cpp` implements first-fit and least-loaded-dominant-resource over feasible workers. The CLI rejects unknown policies and reports CPU/memory deficits when placement is impossible. Scheduler tests cover CPU-heavy, memory-heavy, balanced, infeasible, and worker-ID tie cases. Output: deterministic policy selection shared by the CLI and controller. The exact rule and measured comparison appear in Section 3.

D. Local process supervision. `src/process_supervisor.cpp` and `src/process_supervision_main.cpp` launch children, track PID and lifecycle, stop intentionally, and reap terminated processes. Unexpected exits are distinguished from requested termination, including a child that exits with code zero. Output: `process_supervision_tracer`, JSONL lifecycle events, and `process_supervisor_tests`. Contribution: gives remote launches a real process-management foundation rather than merely recording simulated running states.

E. Worker protocol and discovery. `src/worker_protocol.cpp` implements the TCP message path; the controller registers worker IDs and capacities and receives heartbeats. Controller status exposes health and reservations; events expose registration and heartbeat activity. Output: `worker_protocol_tests` and `controller_worker_integration_tests`. Contribution: turns worker inventory into live network-connected state. Commit b240e2c, 30 September 2026, adds policy, runtime, protocol, controller/worker code, and the five-suite foundation noted in prior feedback.

2. Work completed: Weeks 1-7 (continued)

Cumulative milestones, part B

F. Remote task launch and replicated placement. The controller accepts a policy, CPU/memory request, replica count, and executable. It generates stable workload/replica IDs, reserves capacity before each next decision, dispatches launches, and records worker acknowledgements. Insufficient-capacity replicas stay pending with explicit deficits. Multiple workers are ordered by ID. Output: real remote process launching over localhost TCP, status inspection, replicated_placement_integration_tests, and commit 40cc5dc on 6 October.

G. Explicit stop and accounting. stop <workload-id> changes desired replicas to zero before sending remote stops. Workers terminate/reap supervised processes; reservations are released after confirmed cleanup. Pending replicas stop immediately, and repeated completed stops are harmless. Output: observable PENDING, LAUNCHING, RUNNING, STOPPING, and STOPPED states. Contribution: prevents stopped workloads from being resurrected by reconciliation and makes capacity release traceable.

H. Failure detection and replacement. Connection loss or missed heartbeats marks a worker DEAD, excludes its capacity, marks its replicas LOST, and schedules replacements using the original policy. Recovery events retain IDs, attempt numbers, and timestamps from loss through replacement-running acknowledgement. Default heartbeat timeout is 2000 ms with normal polling at most about 100 ms apart. Tests also use a shorter configured timeout. No-spare-capacity replicas remain lost/pending until capacity arrives. Output: worker_recovery_integration_tests and commit 95a8a6d on 6 October.

I. CPU/memory enforcement. src/cgroup.cpp and the runtime create cgroup v2 scopes, configure cpu.max and memory.max, disable swap, enable group OOM behavior, and attach children before execution. Cleanup includes descendants and is retried on failure; termination is not falsely acknowledged. Launch fails visibly if enforcement is unavailable. Output: cgroup_tests, privileged integration tests, runtime accounting tests, and commit 50d05f0 on 6 October. Actual privileged enforcement tests were skipped in the present environment, so kernel enforcement is implementation evidence rather than a newly measured result.

J. Midterm evidence preparation. A fresh Debug build and nine runnable test suites were executed. A supplemental C++ benchmark calls the existing scheduler directly and records a small paired policy comparison. This benchmark was prepared with AI assistance during report preparation; it is not represented as an earlier student-authored semester deliverable. It provides preliminary evidence while the full experiment issues 009 and 010 remain incomplete.

3. Evidence of progress: architecture and rules

Implemented control and execution path

```

Submission / status / stop
    |
    v
Controller: admission -> scheduler -> reservation ledger
    | TCP launch/stop      ^ registration / heartbeat
    v                      | running / exit acknowledgement
Worker agent -> ProcessSupervisor -> child process
                                   -> per-replica cgroup v2

Missed heartbeat / TCP loss -> DEAD -> LOST replica
    -> replacement placement -> launch -> RUNNING acknowledgement

```

Design evidence: README.md; src/controller_main.cpp; src/worker_agent_main.cpp; include/mini_cloud/process_supervisor.hpp. Simulation does not execute this entire path: it directly calls choose_worker. Integration tests run actual controller/agent binaries and child processes on one Linux host.

Scheduling rule: fully specified

For worker i , let capacities be C_i CPU and M_i memory, existing reservations c_i and m_i , and request $r=(r_c, r_m)$. A candidate must have valid positive capacities, valid reservations, and checked additions. It is feasible exactly when $c_i + r_c \leq C_i$ and $m_i + r_m \leq M_i$. Overflow makes the request infeasible; malformed CLI values are rejected before selection.

```

score_i(r) = max((c_i + r_c) / C_i, (m_i + r_m) / M_i)
first-fit: first feasible worker in candidate order
resource-aware: feasible worker with minimum score_i(r)
exact score tie: smallest worker ID
no feasible worker: no placement; report resource deficits

```

The controller uses worker-ID order; the standalone tracer uses supplied order. Reservations are updated after each accepted replica. src/scheduler.cpp compares fractions using quotient/remainder steps, avoiding floating-point tie ambiguity and overflow from naive cross-products. The scoring policy minimizes projected dominant reserved utilization; it does not measure live CPU usage or solve globally optimal packing.

Reproducible functional examples

```

./build/resource_admission_tracer first-fit 1000 2048 \
  worker-a 2000 4096 500 1024 worker-b 2000 4096 0 0
# selected worker=worker-a

# Controller input after workers register:
submit least-loaded-dominant-resource 500 64 7 -- /bin/sleep 60
status
stop workload-1

```

The README fixture with three 1000m/128MiB workers fits six 500m/64MiB replicas and leaves one pending. First-fit packs a,a,b,b,c,c; the resource-aware policy gives a,b,c,a,b,c. This is a documented deterministic fixture, distinct from the measured benchmark on the next page.

3. Evidence of progress: policy experiment

Executed synthetic scheduler experiment

Harness: `output/evidence/policy_benchmark.cpp` compiles with `src/scheduler.cpp` and `src/resources.cpp` at `-O2`. Four homogeneous simulated workers each have 2000 millicores and 2048 MiB. Worker IDs are `worker-0` through `worker-3`. Requests are placed sequentially without departures; rejection does not change reservations. Both policies receive the same generated stream within each trial.

Matrix: `balanced-light` = 8 requests of (500m,128MiB), initially empty; `mixed-empty` = 48 requests, initially empty; `mixed-preloaded` = the same 48-request generator with each worker pre-reserved at (1000m,1024MiB); `mixed-repeat` = a rerun of `mixed-empty` for reproducibility. Each scenario has 10 trials, trial IDs 0-9 and generator seeds 100-109: 4 scenarios x 2 policies x 10 trials = 80 rows. The `balanced` fixture repeats identical requests; `mixed` scenarios vary deterministic ordering.

The generator updates unsigned 32-bit state as $s = 1664525 * s + 1013904223 \pmod{2^{32}}$, and selects $(s \gg 16) \% 3$: CPU-heavy (800m,64MiB), memory-heavy (100m,768MiB), or mixed-resource (400m,384MiB). This precisely specifies the workload distribution and submission order without claiming measured application demand.

Measures: `placement success` = `placed/requested` x 100%; `resource balance` = population standard deviation across four workers of `reserved/capacity`, expressed in percentage points (pp), separately for CPU and memory; lower SD means more even reservation balance. `Scheduling time` = mean `steady_clock` duration of `choose_worker` per submitted request, excluding reservation update, process launch, and TCP. Table entries average the ten trial results.

Scenario	Policy	Placed	Success	CPU SD	Mem SD	Time us
balanced-light	FF	8.0	100.00%	50.00	12.50	0.0198
balanced-light	LD	8.0	100.00%	0.00	0.00	0.0662
mixed-empty	FF	16.0	33.33%	9.84	10.92	0.0289
mixed-empty	LD	16.2	33.75%	7.12	6.52	0.0527
mixed-preloaded	FF	8.0	16.67%	2.20	1.37	0.0307
mixed-preloaded	LD	8.0	16.67%	2.27	1.42	0.0285
mixed-repeat	FF	16.0	33.33%	9.84	10.92	0.0363
mixed-repeat	LD	16.2	33.75%	7.12	6.52	0.0546

FF = first-fit; LD = least-loaded dominant-resource; SD units = pp.

Interpretation. In the light fixture, both policies place every request; resource-aware scheduling spreads reservations evenly while first-fit concentrates CPU on two workers. In `mixed-empty`, mean success rises from 33.33% to 33.75% (0.42 percentage points) and mean CPU/memory SD decreases. The `preloaded` scenario has equal success and slightly worse balance for resource-aware scheduling, demonstrating that improvement is workload-dependent.

Timing is a preliminary microbenchmark: each trace times only 8 or 48 calls, making clock overhead, caches, and host noise material. The light/mixed-empty measurements suggest extra scoring cost, but `preloaded` timing reverses the ordering. Do not infer a statistically established speed ranking or cluster-wide throughput from these short samples. The `mixed-repeat` rows reproduce every placement-success and balance value from `mixed-empty`; timings vary.

3. Evidence of progress: tests and reproducibility

Fresh build and actual execution

Environment: Linux 7.0.0-34-generic, aarch64; GNU C++ 15.2.0; CMake 4.2.3; Ninja. A clean CMake Debug build completed. Tests initially failed under the restricted sandbox because sockets/child launch were blocked. Rerunning outside that sandbox produced nine passed suites, zero failed suites, and two skipped privileged suites in 8.69 seconds. CTest prints 100% passed, but the accurate denominator is nine executed, not eleven executed.

resource_tests	PASS
scheduler_tests	PASS
process_supervisor_tests	PASS
worker_protocol_tests	PASS
controller_worker_integration_tests	PASS
replicated_placement_integration_tests	PASS
worker_recovery_integration_tests	PASS
cgroup_tests	PASS
runtime_accounting_integration_tests	PASS
cgroup_integration_tests	SKIP (delegation required)
cgroup_worker_integration_tests	SKIP (delegation required)

Coverage: admission fit/overflow invariants; deterministic policy behavior; launch, stop, and child exit; protocol validation; registration/heartbeats and remote launch; multiple-replica placement and stop; agent kill and heartbeat-timeout recovery; no-spare-capacity behavior; cgroup configuration/cleanup with fixtures; terminal-state reservation accounting. These are correctness tests, not a repeated recovery-performance study.

Recovery suite scenarios use real agent processes on localhost, three workers, and three 500m/64MiB sleep replicas. The victim is agent-1, failed only after expected replicas are running. It covers first-fit SIGKILL, resource-aware SIGKILL, resource-aware SIGSTOP/timeout, and a full-capacity recovery case. Event ordering and restoration are checked. These runs do not establish performance on separate physical nodes.

Commands and retained artifacts

```
cmake -S . -B /tmp/mini-cloud-midterm-build -G Ninja \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build /tmp/mini-cloud-midterm-build -j 2
ctest --test-dir /tmp/mini-cloud-midterm-build --output-on-failure
c++ -std=c++23 -O2 -Iinclude output/evidence/policy_benchmark.cpp \
  src/scheduler.cpp src/resources.cpp -o /tmp/mini-cloud-policy-benchmark
/tmp/mini-cloud-policy-benchmark > output/evidence/policy_trials.csv
```

Retained: evidence/ctest.log, policy_trials.csv, policy_benchmark.cpp, environment.json, and build_report.py. Repository access may require instructor permission. An earlier review reported a 3:48 video with accepted/rejected requests and passing tests; that video was not available here and is not independently reverified or substituted for a new midterm presentation.

4. Progress compared with the proposal

Reconstructed plan versus verified progress

Basis: supplied proposal feedback identifies scheduling baselines, failure scenarios, and measurable outcomes; local issues 001-010 supply planned implementation milestones. The approved proposal and semester calendar are absent. This comparison cannot establish exact original due weeks or formally certify that the project is on schedule.

Admission/build foundation (issues 001-002): COMPLETE in code and executed tests. CPU/memory checks, overflow handling, selectable policies, and deterministic decisions form a runnable vertical slice.

Runtime and network foundation (issues 003-005): COMPLETE for the supported scope. Supervision, worker registration, heartbeats, and remote task launch are implemented and backed by executed tests, addressing the areas mentioned in prior deductions.

Replicas, stop/status, and worker recovery (issues 006-007): COMPLETE for agent-crash/TCP-loss and heartbeat-timeout paths. Placement and recovery correctness tests pass. Partition-safe recovery and exactly-once execution are outside the implemented scope.

CPU/memory enforcement (issue 008): IMPLEMENTED; ordinary configuration/cleanup tests pass. Privileged live-kernel enforcement validation is still pending in this evidence snapshot because the required cgroup delegation is unavailable.

Recovery performance study (issue 009): IN PROGRESS. Timestamped recovery events and correctness tests exist, but a versioned trial runner and a repeated quantitative recovery summary are not yet delivered.

Placement-policy simulator/study (issue 010): IN PROGRESS. The report includes a small direct-scheduler comparison with raw observations, explicit workload sizes, seeds, and repeats. A full scenario format, larger timed batches, diagnostic collection, and polished analysis remain unfinished.

Changes and schedule assessment

The visible implementation moved from a narrow admission tracer to end-to-end lifecycle/recovery and cgroup control through small dependency-ordered slices. This is observable progression, not a claim that the approved scope changed. The report adds a preliminary benchmark to resolve the earlier comparison criticism; it does not label the full simulator issue complete.

At the midpoint, core prototype capability is demonstrated, while final evaluation and privileged validation remain open. The second-half plan prioritizes those gaps before adding features. Exact schedule variance must be reconciled with the original proposal when it is available; no unsupported on-time claim is made.

5. Challenges and solutions

Resource correctness. CPU and memory have different units and either can reject an otherwise plausible placement; unsigned sums can overflow. Impact: naive arithmetic could admit impossible reservations. Solution: integer millicores/MiB, checked_add, validation, joint feasibility, and targeted admission tests. Status: resolved for the supported integer request model; runtime usage still differs from declared reservations.

Undefined policy scoring and weak comparison. Prior feedback found the scoring algorithm insufficiently specified and policy comparison unclear. Impact: results could not be reproduced or interpreted. Solution: state the projected dominant-utilization formula, feasibility and tie rules, and run paired identical streams with success, CPU/memory SD, and timing. Status: the rule and preliminary comparison are now explicit; robust latency inference and broader workloads remain open.

Lifecycle and resource-accounting races. Launch failure, natural exit, explicit stop, and cleanup failure can otherwise free capacity too early or twice. Impact: controller state may diverge from running processes. Solution: stable replica IDs, reservation before dispatch, terminal acknowledgements after cleanup, retryable cleanup, and runtime accounting tests. Status: supported transitions pass tests; persistent recovery across controller restart is not established.

Worker failure ambiguity. A dropped connection or silent worker does not prove old workloads stopped. Impact: replacements may execute alongside orphaned originals. Solution: worker-dead/lost/replacement event stages, health exclusion, attempt tracking, and explicit recovery limitations. Status: crash/timeout recovery works in tests; fencing, partition safety, and exactly-once execution remain unresolved and are not claimed.

Kernel permissions and test environment. cgroup enforcement needs a writable delegated parent with cpu/memory controls enabled. Restricted execution also blocked local sockets and child launches. Impact: an initial sandbox test run was not representative, and privileged tests cannot run here. Solution: rerun ordinary tests outside the sandbox, retain the successful log, and reserve kernel enforcement validation for a delegated VM. Status: ordinary execution resolved; privileged enforcement measurements pending.

Release-build portability. An additional -O3 Release attempt failed under -Werror because src/process_supervisor.cpp discards a write() result that the compiler marks warn_unused_result. Impact: this toolchain cannot build that Release configuration unchanged. Solution for report verification: use the successful clean Debug build without modifying project code. Status: Release warning handling remains a specific maintenance target; the benchmark separately compiles scheduler-only code at -O2.

Evidence provenance and timeline. The approved proposal, full weekly log, AS1.pdf, and earlier video were absent. Impact: precise week attribution and original-plan comparison cannot be authenticated from code alone. Solution: group cumulative milestones, cite dated commits, distinguish external feedback from executed evidence, and request the missing proposal. Status: original-document confirmation remains pending.

6. Current status and 7. second-half plan

Midpoint status

Achieved: runnable C++23 controller/worker prototype; validated admission; two deterministic schedulers; child supervision; TCP registration and heartbeats; remote launches; replicated placement; explicit stop/status; agent-failure recovery; implemented cgroup limits; nine passing runnable suites; and a preliminary paired policy comparison. The prototype can schedule and supervise actual local worker processes.

Incomplete: privileged enforcement measurements, repeated recovery timing, a full versioned simulator/analysis pipeline, larger latency batches, cross-node experiments, Release warning correction, and final presentation. Known limits include trusted workloads, possible duplicate execution after agent loss, no fencing/exactly-once guarantee, and no automatic relaunch after healthy-agent child exit. Reported reservations are not measured application utilization.

Scheduled completion targets (relative semester weeks)

Week 8: correct the Release write-result warning and verify Debug/Release builds. Obtain a cpu/memory-delegated Linux VM; execute both privileged suites with zero skips and retain exact cpu.max/memory.max values, throttling evidence, OOM outcome, and cleaned scope paths. Completion: both build modes succeed and two privileged logs are archived.

Week 9: finish issue 009 runner. Save scenario, environment, injection timestamp, and JSONL for each trial. Use three workers of 1000m/128MiB, three replicas of 500m/64MiB, heartbeat 100 ms, timeout 2000 ms. After desired running count stabilizes for two seconds, inject SIGKILL or SIGSTOP into the worker hosting replica 1. Completion: one-command runner and independently recomputed smoke-trial durations.

Week 10: execute recovery matrix: two policies x two failure modes x ten trials = 40 spare-capacity trials. Add two policies x ten SIGKILL trials with three 500m/64MiB workers fully occupied = 20 no-spare trials. In no-spare trials, register a new 1000m/128MiB worker exactly five seconds after failure. Completion: all 60 trial records retained, or failures explicitly reported; median/range for detection, reconciliation, and restoration.

Week 11: finish issue 010 scenario/analysis tooling. Four workers of 2000m/2048MiB; request counts 8,48,96; empty/50%-reserved start; generated/reversed order; seeds 100-109; both policies. Matrix: $3 \times 2 \times 2 \times 10 \times 2 = 240$ runs. Use at least 100,000 scheduler calls in resettable batches for timing. Completion: identical paired streams, deterministic aggregate checks, CSV, and success/balance/time plots.

Week 12: repeat a selected recovery subset on three separate Linux worker nodes if hardware is available; retain host/transport metadata. Otherwise label all results single-host and record this scope change. Completion: provenance is explicit for every figure. Week 13: integrate findings, reproduce one result from raw data, document operational limits. Week 14: record a 3-5 minute live demonstration and submit final artifacts. Completion: reproducible report, working repository link, and playable video.

Evaluation definitions and midterm presentation

Planned recovery measures and interpretation

For each recovery trial, preserve t_{inject} from the runner, t_{dead} from `worker_dead`, t_{issue} from `replacement_launch_issued`, and t_{restored} when the desired acknowledged running count is reestablished. $\text{Detection} = t_{\text{dead}} - t_{\text{inject}}$; $\text{controller reconciliation} = t_{\text{issue}} - t_{\text{dead}}$; $\text{end-to-end restoration} = t_{\text{restored}} - t_{\text{inject}}$. Keep all clocks on the controller/runner host, or document synchronization for cross-host timing. Separate deliberate five-second capacity wait in no-spare trials from scheduling cost.

Report trial-level values and median/range for ten-trial cells; do not present an unstable p95 as a strong finding. Pair policies by scenario/seed. Failures and missing event boundaries are reported rather than discarded. Success is defined using acknowledged replacements, not proof that orphaned old processes terminated. Resource-balance plots use reservation ratios unless kernel usage is independently sampled.

3-5 minute midterm video: approximately 4:20

0:00-0:30: introduce Mini Cloud, the CPU/memory placement problem, and the expected prototype and systems-study outcome. Show the repository and briefly explain the controller/worker diagram.

0:30-1:10: demonstrate `resource_admission_tracer` with one accepted and one rejected request; explain millicores/MiB, overflow checks, and the exact resource-aware formula. Show both policy names and why projected utilization matters.

1:10-2:15: show actual controller and worker terminals. Register three workers, inspect heartbeat/status output, submit replicated `/bin/sleep` workloads, show placement/running events and pending capacity deficits, then demonstrate stop and reservation release. Label `--no-cgroups` explicitly if the demonstration lacks delegation.

2:15-3:05: use a controlled demo environment to kill one hosting agent after workloads stabilize. Show `worker_dead`, `replica_lost`, replacement launch, and restored running count. Explain possible duplicate execution and distinguish single-host processes from separate worker machines.

3:05-3:45: show the retained test log: nine passed and two privileged skips. Show the policy-comparison table, explaining placement success, reservation SD, and why the timing sample is preliminary. Avoid presenting skipped enforcement as verified.

3:45-4:20: explain the Week 8-14 targets: delegated enforcement validation, repeated recovery matrix, larger simulator comparison, cross-node provenance, and final deliverables. End with the repository link.

Submission and attribution

Submit `Midterm_Project_Report.pdf`, the recorded 3-5 minute video, and the repository link to Canvas. The PDF and accompanying script do not themselves constitute a recorded video. This report was prepared with AI assistance from local source code, supplied review comments, and newly executed checks; confirm the inferred name, original proposal timeline, and individual contributions before submission. No grading outcome is guaranteed.